

## ● Information Gathering

### 1. Manually Explore the Site

#### ● Steps:

- Open the website in your browser.
  - Navigate through all visible links, buttons, and pages.
  - Take notes on the functionality, features, and potential areas of interest (e.g., login pages, search bars, forms).
  - Look for hidden or less obvious features (e.g., right-click menus, keyboard shortcuts).
  - Check for client-side behavior (e.g., JavaScript-heavy functionality).
- 

### 2. Spider/Crawl for Missed or Hidden Content

#### ● Steps:

- Use tools like **Burp Suite**, **OWASP ZAP**, or **Arachni** to spider/crawl the site.
  - Configure the tool to follow all links and discover hidden directories or files.
  - Review the results for any unusual or unexpected endpoints.
  - Manually verify any interesting findings.
- 

### 3. Check for Files That Expose Content

#### ● Steps:

- Access common files like:
    - `robots.txt` (e.g., `https://example.com/robots.txt`)
    - `sitemap.xml` (e.g., `https://example.com/sitemap.xml`)
    - `.DS_Store` (e.g., `https://example.com/.DS_Store`)
    - `.git/` or `.svn/` directories (e.g., `https://example.com/.git/`)
  - Use tools like **DirBuster**, **GoBuster**, or **ffuf** to brute-force directories and files.
  - Look for sensitive information or hidden endpoints in these files.
-

## 4. Check the Caches of Major Search Engines

- **Steps:**
    - Use Google's cache feature by searching `cache:<URL>` (e.g., `cache:example.com`).
    - Use tools like **Wayback Machine** (<https://archive.org/web/>) to view historical snapshots of the site.
    - Look for outdated or sensitive information that may have been removed but is still accessible.
- 

## 5. Check for Differences in Content Based on User Agent

- **Steps:**
    - Use browser developer tools (F12) or tools like **Burp Suite** to modify the `User-Agent` header.
    - Test with different user agents (e.g., mobile, search engine crawlers).
    - Compare the responses to identify differences in content or functionality.
    - Look for hidden features or vulnerabilities specific to certain user agents.
- 

## 6. Perform Web Application Fingerprinting

- **Steps:**
    - Use tools like **Wappalyzer**, **BuiltWith**, or **WhatWeb** to identify the technologies used (e.g., CMS, frameworks, server software).
    - Manually inspect HTTP headers, cookies, and source code for clues.
    - Identify the server type (e.g., Apache, Nginx), programming languages (e.g., PHP, Python), and frameworks (e.g., React, Angular).
- 

## 7. Identify Technologies Used

- **Steps:**
  - Use the information gathered from fingerprinting tools.
  - Check for JavaScript libraries, APIs, and third-party integrations.

- Look for outdated or vulnerable versions of software.
- 

## 8. Identify User Roles

- **Steps:**
    - Explore the site for different user roles (e.g., guest, user, admin).
    - Create test accounts for each role if possible.
    - Compare permissions and functionality between roles.
    - Look for privilege escalation opportunities.
- 

## 9. Identify Application Entry Points

- **Steps:**
    - List all input fields (e.g., login forms, search bars, file uploads).
    - Identify APIs, endpoints, and parameters used by the application.
    - Check for hidden or undocumented endpoints.
- 

## 10. Identify Client-Side Code

- **Steps:**
    - Use browser developer tools to inspect the JavaScript, HTML, and CSS.
    - Look for sensitive information (e.g., API keys, tokens) hardcoded in the client-side code.
    - Analyze the behavior of JavaScript functions for potential vulnerabilities (e.g., DOM-based XSS).
- 

## 11. Identify Multiple Versions/Channels

- **Steps:**
  - Check for different versions of the site (e.g., `m.example.com`, `api.example.com`).
  - Look for mobile apps and inspect their network traffic using tools like **Burp Suite** or **Fiddler**.
  - Identify web services (e.g., REST APIs, SOAP) and test them separately.

---

## 12. Identify Co-Hosted and Related Applications

- **Steps:**
    - Use tools like **Sublist3r**, **Amass**, or **Censys** to find subdomains and related domains.
    - Check for shared infrastructure (e.g., IP ranges, SSL certificates).
    - Look for connections between the main site and other applications.
- 

## 13. Identify All Hostnames and Ports

- **Steps:**
    - Use tools like **Nmap** or **Masscan** to scan for open ports and services.
    - Perform DNS enumeration to find all hostnames associated with the target.
    - Check for misconfigured services (e.g., open databases, FTP servers).
- 

## 14. Identify Third-Party Hosted Content

- **Steps:**
    - Inspect the site for third-party scripts, libraries, or content (e.g., Google Analytics, jQuery).
    - Use tools like **Burp Suite** or **OWASP ZAP** to monitor network traffic.
    - Check for vulnerabilities in third-party components.
- 

## Tools to Use

- **Information Gathering:**
  - **Sublist3r**, **Amass**, **Censys** (subdomains and related apps)
  - **Nmap**, **Masscan** (hostnames and ports)
  - **Wappalyzer**, **BuiltWith**, **WhatWeb** (technologies)
- **Crawling/Spidering:**
  - **Burp Suite**, **OWASP ZAP**, **Arachni**
- **Directory Brute-Forcing:**
  - **DirBuster**, **GoBuster**, **ffuf**

- **Fingerprinting:**
  - Wappalyzer, WhatWeb, BuiltWith
- **Traffic Analysis:**
  - Burp Suite, Fiddler, Charles Proxy

Ai agent :

infoGathering.py

---

## ● Configuration Management

### 1. Check for Commonly Used Application and Administrative URLs

- **Objective:** Identify default or predictable URLs that may expose sensitive functionality.
  - **Steps:**
    1. Use tools like **dirsearch**, **gobuster**, or **ffuf** to enumerate directories and files.
    2. Look for common admin paths like `/admin`, `/login`, `/dashboard`, `/wp-admin`, `/manager`, etc.
    3. Check for default URLs of frameworks (e.g., `/phpmyadmin`, `/jenkins`, `/grafana`).
    4. Use Google Dorks or search engines to find exposed administrative interfaces.
    5. Verify if these URLs are accessible and if they expose sensitive functionality.
- 

### 2. Check for Old, Backup, and Unreferenced Files

- **Objective:** Identify outdated or forgotten files that may contain sensitive information.
- **Steps:**
  1. Use tools like **dirsearch** or **gobuster** with a wordlist for common backup extensions (e.g., `.bak`, `.old`, `.zip`, `.tar`, `.sql`).
  2. Look for version control files (e.g., `.git`, `.svn`, `.hg`) using tools like **GitHacker** or **dvcs-ripper**.
  3. Manually check for unreferenced files in the source code or by analyzing network traffic.

4. Verify if these files contain sensitive data like credentials, API keys, or source code.
- 

### 3. Check HTTP Methods Supported and Cross-Site Tracing (XST)

- **Objective:** Identify potentially dangerous HTTP methods and test for XST vulnerabilities.
  - **Steps:**
    1. Use tools like **Nmap** (`nmap --script http-methods`) or **curl** to enumerate supported HTTP methods:

```
2. bash
3. Copy
```
    4. `curl -X` `-I`
    5. Check if dangerous methods like `PUT`, `DELETE`, or `TRACE` are enabled.
    6. Test for Cross-Site Tracing (XST) by sending a `TRACE` request and checking if the response reflects the input:

```
7. bash
8. Copy
```
    9. `curl -X` `-H "Test: XST"`
    10. If `TRACE` is enabled, verify if it can be exploited to steal cookies or sensitive data.
- 

### 4. Test File Extensions Handling

- **Objective:** Identify how the server handles different file extensions and if it exposes sensitive files.
  - **Steps:**
    1. Test for file extensions like `.php`, `.jsp`, `.asp`, `.html`, `.json`, `.xml`, etc.
    2. Upload files with different extensions (if allowed) and check how they are processed.
    3. Check if the server returns sensitive files (e.g., `.git`, `.env`, `.htaccess`).
    4. Test for improper handling of extensions (e.g., `.php.txt` or `.php%00.jpg`).
-

## 5. Test for Security HTTP Headers

- **Objective:** Verify if the application uses security headers to protect against common attacks.
  - **Steps:**
    1. Use browser developer tools or tools like **curl** to inspect HTTP headers:
    2. `bash`
    3. Copy
    4. `curl -I`
    5. Check for the following headers:
      - **Content Security Policy (CSP):** Prevents XSS and data injection attacks.
      - **X-Frame-Options:** Prevents clickjacking attacks.
      - **Strict-Transport-Security (HSTS):** Enforces HTTPS.
      - **X-Content-Type-Options:** Prevents MIME sniffing.
      - **Referrer-Policy:** Controls referrer information.
    6. Verify if headers are missing or misconfigured.
- 

## 6. Test for Policies (e.g., Flash, Silverlight, robots)

- **Objective:** Identify policies that may expose sensitive information or functionality.
  - **Steps:**
    1. Check the `robots.txt` file for disallowed paths that may still be accessible.
    2. Look for Flash or Silverlight policies (e.g., `crossdomain.xml`, `clientaccesspolicy.xml`) and verify if they allow unauthorized access.
    3. Test if these policies are overly permissive or expose sensitive endpoints.
- 

## 7. Test for Non-Production Data in Live Environment, and Vice-Versa

- **Objective:** Identify if non-production data (e.g., test data, staging data) is exposed in the live environment.
- **Steps:**
  1. Look for endpoints or files that reference test or staging environments.
  2. Check for hardcoded credentials, test API keys, or dummy data in the live environment.
  3. Use tools like **Burp Suite** or **ZAP** to analyze responses for non-production data.

4. Verify if the live environment is accidentally serving non-production data.
- 

## 8. Check for Sensitive Data in Client-Side Code

- **Objective:** Identify sensitive data (e.g., API keys, credentials) exposed in client-side code.
  - **Steps:**
    1. Inspect the source code of the application using browser developer tools.
    2. Look for hardcoded credentials, API keys, or tokens in JavaScript files.
    3. Use tools like **GitHub Dorks** or **truffleHog** to search for sensitive data in publicly accessible repositories.
    4. Check for comments in HTML, CSS, or JavaScript files that may contain sensitive information.
- 

### Tools to Use:

- **Enumeration:** dirsearch, gobuster, ffuf, Nmap
- **HTTP Analysis:** curl, Burp Suite, ZAP
- **Source Code Analysis:** browser developer tools, truffleHog
- **Header Analysis:** SecurityHeaders.com, curl

Ai agent:

`ai_config_management_agent.py`

`#python3 ai_config_management_agent.py --url https://example.com`

---

- **Secure Transmission**

### 1. Check SSL Version, Algorithms, Key Length

- **Objective:** Ensure the server uses secure SSL/TLS versions and strong cryptographic algorithms.
- **Steps:**

1. Use tools like **SSL Labs (SSLLabs.com)** or **testssl.sh** to analyze the server's SSL/TLS configuration:  
`./testssl.sh example.com`
  2. Verify that insecure protocols (SSLv2,
  3. SSLv3, TLS 1.0, TLS 1.1) are disabled.
  4. Check that strong encryption algorithms are used (e.g., AES, ChaCha20) and weak algorithms (e.g., RC4, DES) are disabled.
  5. Ensure the key length is at least 2048 bits for RSA or 256 bits for ECC.
- 

## 2. Check for Digital Certificate Validity (Duration, Signature, and CN)

- **Objective:** Verify that the SSL/TLS certificate is valid and properly configured.
  - **Steps:**
    1. Use tools like **SSL Labs** or **openssl** to inspect the certificate:  
`openssl s_client -connect example.com:443 | openssl x509 -text -noout`
    2. Check the **expiration date** to ensure the certificate is not expired or close to expiry.
    3. Verify the **Common Name (CN)** or **Subject Alternative Names (SANs)** match the domain name.
    4. Check the **signature algorithm** (e.g., SHA-256) to ensure it is secure.
    5. Look for certificate transparency logs to ensure the certificate is legitimate.
- 

## 3. Check Credentials Only Delivered Over HTTPS

- **Objective:** Ensure credentials (e.g., usernames, passwords) are only transmitted over HTTPS.
  - **Steps:**
    1. Use a proxy tool like **Burp Suite** or **OWASP ZAP** to intercept login requests.
    2. Verify that the login form submission URL uses `https://` and not `http://`.
    3. Check for mixed content issues where HTTP is used on an HTTPS page.
    4. Test for insecure redirects (e.g., HTTP → HTTPS) that may expose credentials.
-

#### 4. Check That the Login Form Is Delivered Over HTTPS

- **Objective:** Ensure the login page itself is served over HTTPS.
  - **Steps:**
    1. Navigate to the login page and check the URL in the browser's address bar.
    2. Verify that the URL begins with `https://` and that the certificate is valid.
    3. Use browser developer tools to check for mixed content (e.g., HTTP resources on an HTTPS page).
    4. Test for insecure redirects (e.g., HTTP → HTTPS) that may expose the login form.
- 

#### 5. Check Session Tokens Only Delivered Over HTTPS

- **Objective:** Ensure session tokens (e.g., cookies) are only transmitted over HTTPS.
  - **Steps:**
    1. Use a proxy tool like **Burp Suite** or **OWASP ZAP** to intercept requests and responses.
    2. Check if session cookies have the **Secure** flag set (e.g., `Set-Cookie: sessionId=123; Secure`).
    3. Verify that session tokens are not transmitted over HTTP.
    4. Test for insecure redirects that may expose session tokens.
- 

#### 6. Check if HTTP Strict Transport Security (HSTS) Is in Use

- **Objective:** Ensure the server enforces HTTPS using HSTS.
- **Steps:**
  1. Use tools like **SSL Labs** or **curl** to check for the `Strict-Transport-Security` header:  

```
curl -I https://example.com
```
  2. Verify that the header includes a `max-age` directive (e.g., `max-age=31536000`).
  3. Check if the `includeSubDomains` directive is present to enforce HTTPS on subdomains.
  4. Test if the server redirects HTTP requests to HTTPS (e.g., `http://example.com` → `https://example.com`).

---

## Tools to Use:

- **SSL/TLS Analysis:** SSL Labs (SSLLabs.com), testssl.sh, openssl
- **Proxy Tools:** Burp Suite, OWASP ZAP
- **Browser Developer Tools:** Chrome DevTools, Firefox Developer Tools

Ai agent:

secure\_transmission\_agent.py

python secure\_transmission\_agent.py --domain example.com

---

## ● Authentication

### 1. Test for User Enumeration

- **Objective:** Determine if an attacker can identify valid usernames or email addresses.
  - **Steps:**
    1. Attempt to register with a known username/email and observe error messages (e.g., "Username already exists").
    2. Use the login form to test for differences in error messages (e.g., "Invalid username" vs. "Invalid password").
    3. Test the password reset functionality for similar behavior.
    4. Use tools like **Burp Suite** or **OWASP ZAP** to automate testing with a list of common usernames/emails.
- 

### 2. Test for Authentication Bypass

- **Objective:** Identify ways to bypass authentication mechanisms.
- **Steps:**
  1. Test for SQL injection in the login form (e.g., ' OR '1'='1').
  2. Manipulate session cookies or tokens to gain unauthorized access.
  3. Check for insecure direct object references (IDOR) in URLs or parameters.

4. Test for missing or weak authorization checks on authenticated endpoints.
- 

### 3. Test for Brute Force Protection

- **Objective:** Verify if the application prevents brute force attacks.
  - **Steps:**
    1. Attempt multiple failed login attempts and observe if the account is locked or CAPTCHA is triggered.
    2. Check for rate-limiting mechanisms (e.g., delays or IP blocking).
    3. Test if error messages reveal whether the account is locked or not.
    4. Use tools like **Hydra** or **Burp Intruder** to automate brute force testing.
- 

### 4. Test Password Quality Rules

- **Objective:** Ensure the application enforces strong password policies.
  - **Steps:**
    1. Attempt to create or change passwords with weak patterns (e.g., 123456, password).
    2. Verify if the application enforces minimum length, complexity, and uniqueness.
    3. Test for password reuse policies (e.g., preventing the use of previous passwords).
    4. Check if the application provides feedback on password strength.
- 

### 5. Test Remember Me Functionality

- **Objective:** Ensure the "Remember Me" feature is implemented securely.
  - **Steps:**
    1. Enable "Remember Me" and inspect the generated cookie.
    2. Verify if the cookie is persistent and contains sensitive data (e.g., username, password).
    3. Test if the cookie is invalidated after logout or password change.
    4. Check if the cookie is encrypted or signed to prevent tampering.
-

## 6. Test for Autocomplete on Password Forms/Input

- **Objective:** Ensure password fields disable autocomplete to prevent browser caching.
  - **Steps:**
    1. Inspect the login form's HTML for the `autocomplete="off"` attribute.
    2. Test if the browser suggests previously saved passwords.
    3. Verify if the password field is marked as sensitive (e.g., `<input type="password">`).
- 

## 7. Test Password Reset and/or Recovery

- **Objective:** Identify vulnerabilities in the password reset process.
  - **Steps:**
    1. Test if the reset link is sent over HTTPS.
    2. Verify if the reset token is sufficiently random and expires after use.
    3. Check for insecure practices like token reuse or predictable tokens.
    4. Test if the reset functionality is vulnerable to user enumeration or brute force attacks.
- 

## 8. Test Password Change Process

- **Objective:** Ensure the password change process is secure.
  - **Steps:**
    1. Verify if the current password is required to change the password.
    2. Test if the new password adheres to the application's password policy.
    3. Check if the session is invalidated after a password change.
    4. Test for CSRF vulnerabilities in the password change form.
- 

## 9. Test CAPTCHA

- **Objective:** Ensure CAPTCHA is implemented effectively to prevent automation.
- **Steps:**
  1. Test if the CAPTCHA can be bypassed (e.g., by reusing tokens or disabling JavaScript).
  2. Verify if the CAPTCHA is triggered after multiple failed attempts.

3. Check if the CAPTCHA is user-friendly and accessible.
  4. Use tools like **Burp Suite** to automate CAPTCHA bypass attempts.
- 

## 10. Test Multi-Factor Authentication (MFA)

- **Objective:** Verify the effectiveness of MFA mechanisms.
  - **Steps:**
    1. Test if MFA is enforced for sensitive actions (e.g., login, password change).
    2. Verify if the MFA token is delivered securely (e.g., via HTTPS or app-based).
    3. Check if the MFA implementation is vulnerable to bypass (e.g., reusing tokens).
    4. Test for rate-limiting or lockout mechanisms on MFA attempts.
- 

## 11. Test for Logout Functionality Presence

- **Objective:** Ensure the logout functionality works as expected.
  - **Steps:**
    1. Verify if the logout button is present and functional.
    2. Test if the session is invalidated after logout.
    3. Check if the browser cache is cleared after logout.
    4. Verify if the user is redirected to a secure page after logout.
- 

## 12. Test for Cache Management on HTTP (e.g., Pragma, Expires, Max-age)

- **Objective:** Ensure sensitive data is not cached by the browser.
  - **Steps:**
    1. Inspect HTTP headers for caching directives (e.g., `Cache-Control: no-store`, `Pragma: no-cache`).
    2. Verify if sensitive pages (e.g., login, account settings) are excluded from caching.
    3. Test if the browser caches sensitive data (e.g., passwords, session tokens).
-

### 13. Test for Default Logins

- **Objective:** Identify default or hardcoded credentials.
  - **Steps:**
    1. Check for default usernames/passwords (e.g., `admin:admin`).
    2. Test for common credentials using tools like **Hydra** or **Burp Intruder**.
    3. Verify if the application allows changing default credentials.
- 

### 14. Test for User-Accessible Authentication History

- **Objective:** Ensure users can view their authentication history.
  - **Steps:**
    1. Check if the application provides a log of recent logins (e.g., IP address, timestamp).
    2. Verify if users can revoke active sessions.
    3. Test if the history is stored securely and not exposed to unauthorized users.
- 

### 15. Test for Out-of-Channel Notification of Account Lockouts and Successful Password Changes

- **Objective:** Ensure users are notified of critical account events.
  - **Steps:**
    1. Test if users receive email or SMS notifications for account lockouts.
    2. Verify if users are notified of successful password changes.
    3. Check if the notifications contain sensitive information (e.g., old password).
- 

### 16. Test for Consistent Authentication Across Applications with Shared Authentication Schema / SSO

- **Objective:** Ensure consistent authentication across integrated systems.
- **Steps:**
  1. Test if logging into one application grants access to others.
  2. Verify if logout from one application logs out from all.

3. Check for vulnerabilities in the SSO implementation (e.g., insecure token handling).
- 

## 17. Registration

- **Objective:** Ensure the registration process is secure.
  - **Steps:**
    1. Test if the registration form is delivered over HTTPS.
    2. Verify if the application enforces strong password policies during registration.
    3. Check for user enumeration during registration.
    4. Test if the registration process is vulnerable to automation (e.g., CAPTCHA bypass).
- 

### Tools to Use:

- **Proxy Tools:** Burp Suite, OWASP ZAP
- **Brute Force Tools:** Hydra, Burp Intruder
- **SSL/TLS Analysis:** SSL Labs, testssl.sh
- **Browser Developer Tools:** Chrome DevTools, Firefox Developer Tools

Ai agent:

authentication\_agent.py

```
python authentication_agent.py --login_url https://example.com/login --registration_url  
https://example.com/register --password_reset_url https://example.com/reset --logout_url  
https://example.com/logout
```

---

- Session Management

## 1. Establish How Session Management Is Handled in the Application

- **Objective:** Understand how the application manages sessions (e.g., cookies, URL tokens).
  - **Steps:**
    1. Inspect the application's login process to identify how session tokens are generated and stored.
    2. Check if session tokens are stored in cookies, URLs, or local storage.
    3. Use browser developer tools or proxy tools like **Burp Suite** to analyze session tokens.
- 

## 2. Check Session Tokens for Cookie Flags (HttpOnly and Secure)

- **Objective:** Ensure session cookies are protected against client-side attacks.
  - **Steps:**
    1. Inspect the session cookie using browser developer tools or proxy tools.
    2. Verify if the **HttpOnly** flag is set to prevent JavaScript access.
    3. Verify if the **Secure** flag is set to ensure the cookie is only transmitted over HTTPS.
    4. Test if the application enforces these flags consistently.
- 

## 3. Check Session Cookie Scope (Path and Domain)

- **Objective:** Ensure session cookies are scoped properly to prevent unauthorized access.
  - **Steps:**
    1. Inspect the **Path** attribute of the session cookie to ensure it is restricted to the application's root or specific paths.
    2. Verify if the **Domain** attribute is set correctly to prevent the cookie from being sent to subdomains or unrelated domains.
    3. Test if the cookie is accessible from unauthorized paths or domains.
- 

## 4. Check Session Cookie Duration (Expires and Max-Age)

- **Objective:** Ensure session cookies have a reasonable expiration time.
- **Steps:**
  1. Inspect the **Expires** or **Max-Age** attributes of the session cookie.

2. Verify if the session duration is appropriate for the application's use case (e.g., short for banking, longer for social media).
  3. Test if the session remains active beyond the specified duration.
- 

## 5. Check Session Termination After a Maximum Lifetime

- **Objective:** Ensure sessions are terminated after a maximum lifetime.
  - **Steps:**
    1. Log in and wait for the session to expire based on the application's maximum lifetime.
    2. Verify if the session is invalidated and the user is forced to re-authenticate.
    3. Test if the session remains active beyond the maximum lifetime.
- 

## 6. Check Session Termination After Relative Timeout

- **Objective:** Ensure sessions are terminated after a period of inactivity.
  - **Steps:**
    1. Log in and remain inactive for the application's timeout period.
    2. Verify if the session is invalidated and the user is forced to re-authenticate.
    3. Test if the session remains active after the timeout period.
- 

## 7. Check Session Termination After Logout

- **Objective:** Ensure sessions are terminated when the user logs out.
  - **Steps:**
    1. Log in and then log out of the application.
    2. Verify if the session token is invalidated and cannot be reused.
    3. Test if the session remains active after logout.
- 

## 8. Test to See If Users Can Have Multiple Simultaneous Sessions

- **Objective:** Ensure the application handles multiple sessions securely.
- **Steps:**

1. Log in from multiple devices or browsers using the same credentials.
  2. Verify if the application allows multiple sessions or enforces a single session policy.
  3. Test if the application provides a way to view or terminate active sessions.
- 

## 9. Test Session Cookies for Randomness

- **Objective:** Ensure session tokens are sufficiently random to prevent prediction.
  - **Steps:**
    1. Collect multiple session tokens and analyze them for patterns.
    2. Use tools like **Burp Sequencer** or **ent** to test the randomness of session tokens.
    3. Verify if the tokens are long and complex enough to resist brute force attacks.
- 

## 10. Confirm That New Session Tokens Are Issued on Login, Role Change, and Logout

- **Objective:** Ensure session tokens are regenerated during critical actions.
  - **Steps:**
    1. Log in and verify if a new session token is issued.
    2. Change the user's role (e.g., from user to admin) and verify if a new token is issued.
    3. Log out and verify if the session token is invalidated.
- 

## 11. Test for Consistent Session Management Across Applications with Shared Session Management

- **Objective:** Ensure consistent session handling across integrated systems.
- **Steps:**
  1. Test if logging into one application grants access to others with shared session management.
  2. Verify if logout from one application logs out from all.
  3. Check for vulnerabilities in the shared session management implementation.

---

## 12. Test for Session Puzzling

- **Objective:** Identify if the application is vulnerable to session puzzling (e.g., using the same session token for multiple purposes).
  - **Steps:**
    1. Test if the same session token is used across different functionalities (e.g., login, password reset).
    2. Verify if the application properly segregates session tokens for different purposes.
    3. Check for unexpected behavior when reusing session tokens.
- 

## 13. Test for CSRF and Clickjacking

- **Objective:** Ensure the application is protected against CSRF and clickjacking attacks.
  - **Steps:**
    - **CSRF:**
      1. Test if sensitive actions (e.g., password change, fund transfer) are protected by CSRF tokens.
      2. Verify if the CSRF token is unique per session and tied to the user's session.
      3. Test if the application validates the CSRF token on the server side.
    - **Clickjacking:**
      1. Test if the application uses the `X-Frame-Options` header to prevent framing.
      2. Verify if the `Content-Security-Policy (CSP)` header includes `frame-ancestors` to restrict framing.
      3. Use tools like **Burp Suite** or browser developer tools to test for clickjacking vulnerabilities.
- 

### Tools to Use:

- **Proxy Tools:** Burp Suite, OWASP ZAP
- **Cookie Analysis:** Browser developer tools, Cookie Editor
- **Randomness Testing:** Burp Sequencer, ent
- **CSRF Testing:** Burp Suite, CSRF PoC generators

Ai agent:

session\_management\_agent.py

```
python session_management_agent.py --login_url https://example.com/login --logout_url https://example.com/logout
```

---

- Authorization

### 1. Test for Path Traversal

- **Objective:** Verify if the application is vulnerable to unauthorized file access.
  - **Steps:**
    1. Test for directory traversal by manipulating file paths in URLs or parameters (e.g., `../../../../etc/passwd`).
    2. Use tools like **Burp Suite** or **OWASP ZAP** to automate path traversal testing.
    3. Check if the application sanitizes user input to prevent traversal attacks.
    4. Verify if sensitive files (e.g., configuration files, logs) are accessible.
- 

### 2. Test for Bypassing Authorization Schema

- **Objective:** Identify ways to bypass authorization checks.
  - **Steps:**
    1. Test for missing or weak authorization checks on sensitive endpoints.
    2. Manipulate parameters (e.g., `user_id`, `role`) to access unauthorized resources.
    3. Use tools like **Burp Suite** to test for insecure direct object references (IDOR).
    4. Verify if the application enforces authorization at both the UI and server levels.
- 

### 3. Test for Vertical Access Control Problems (Privilege Escalation)

- **Objective:** Ensure users cannot escalate their privileges (e.g., user to admin).

- **Steps:**
    1. Log in as a low-privilege user and attempt to access high-privilege functionality.
    2. Test for missing or weak authorization checks on admin endpoints.
    3. Manipulate parameters (e.g., `role=admin`) to escalate privileges.
    4. Verify if the application enforces role-based access control (RBAC) properly.
- 

#### 4. Test for Horizontal Access Control Problems

- **Objective:** Ensure users cannot access resources belonging to other users at the same privilege level.
  - **Steps:**
    1. Log in as User A and attempt to access resources belonging to User B (e.g., `user_id=123` → `user_id=456`).
    2. Test for insecure direct object references (IDOR) in URLs or parameters.
    3. Verify if the application enforces proper ownership checks.
    4. Use tools like **Burp Suite** to automate horizontal access control testing.
- 

#### 5. Test for Missing Authorization

- **Objective:** Identify endpoints or functionality that lack authorization checks.
  - **Steps:**
    1. Test if sensitive endpoints (e.g., admin panels, API endpoints) are accessible without authentication.
    2. Verify if the application enforces authorization for all sensitive actions (e.g., password change, data deletion).
    3. Use tools like **Burp Suite** or **OWASP ZAP** to spider the application and identify unprotected endpoints.
    4. Check if the application provides proper error messages for unauthorized access attempts.
- 

#### Tools to Use:

- **Proxy Tools:** Burp Suite, OWASP ZAP
- **Automated Scanners:** Nessus, Acunetix

- **Custom Scripts:** Python, Bash

Ai agent:

authorization\_agent.py

```
python authorization_agent.py --base_url https://example.com --user_url  
https://example.com/user --admin_url https://example.com/admin --user1_url  
https://example.com/user1 --user2_url https://example.com/user2 --protected_url  
https://example.com/protected
```